Анализ и концептуальное моделирование систем

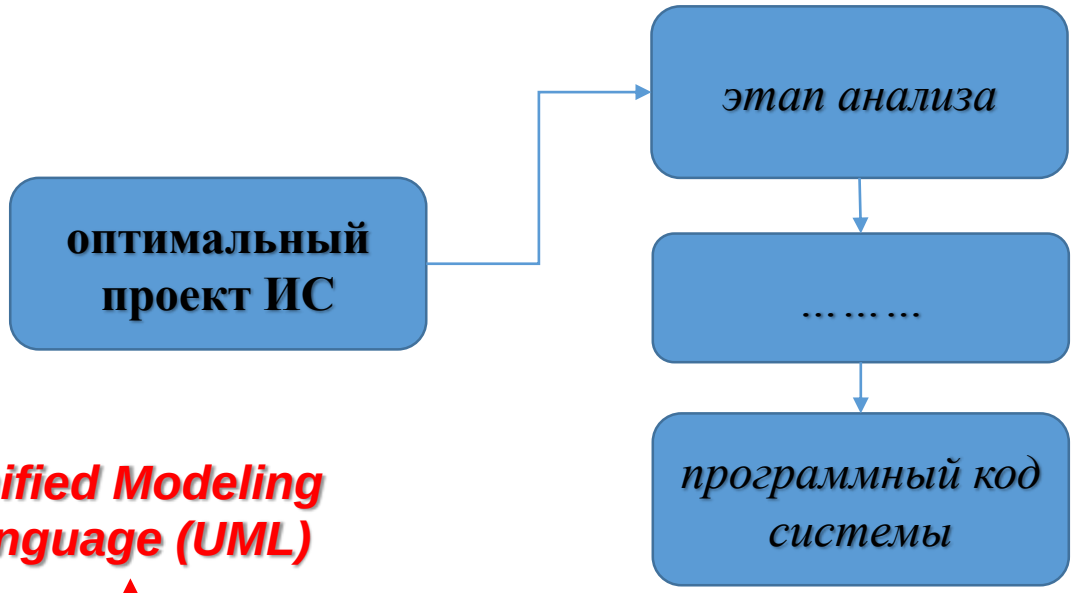
профессор, д.т.н. Смоленцева Т.Е.:
smolenceva@mirea.ru:

ЛЕКЦИЯ 4.
Основы унифицированного языка моделирования.

СОДЕРЖАНИЕ ЛЕКЦИИ:

- Унифицированный язык объектно-ориентированного моделирования.
- Синтаксис и семантика основных объектов UML.
- Понятие модели и диаграмм.

1. Унифицированный язык объектно-ориентированного моделирования



проблема организации взаимодействия между различными командами, реализующими проект

Unified Modeling Language (UML)

средство достижения компромисса

распространение объектно-ориентированных языков программирования в конце 1980-х — начале 1990-х годов

К середине девяностых методы Booch (Grady Booch), OMT-2 (Jim Rumbaugh), OOSE — Object-Oriented Software Engineering (Ivar Jacobson)

OOSE был лучшим на стадии анализа проблемной области и анализа требований к системе, OMT-2 был наиболее предпочтителен на стадиях анализа и разработки информационных систем, Booch лучше всего подходил для стадий дизайна и разработки

1994 г., когда Джим Рамбо и Гради Буч из Rational Software Corporation стали работать над объединением своих методов OMT и Booch. Осенью 1995 г. увидела свет первая черновая версия объединенной методологии, которую они назвали Unified Method 0.8.

UML

характеристики

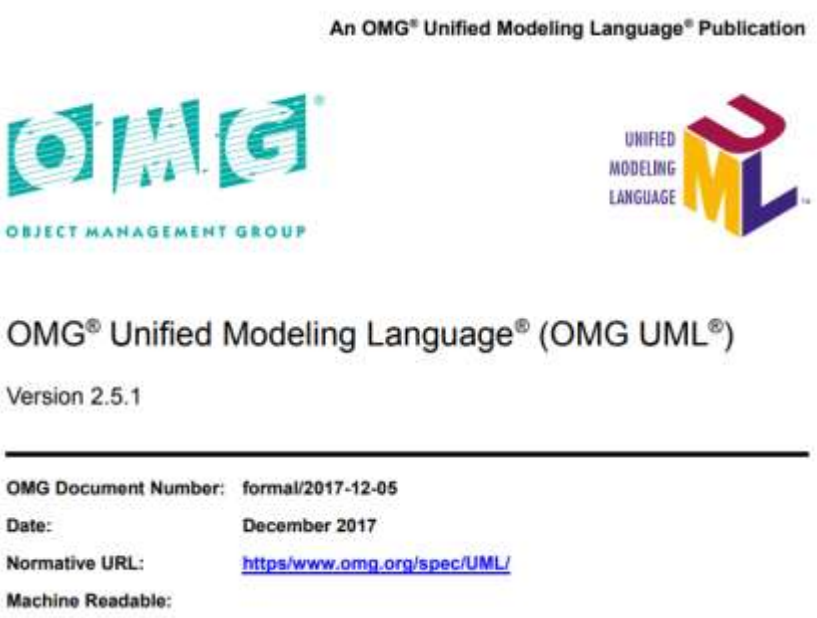
- является языком визуального моделирования, который обеспечивает разработку репрезентативных моделей для организации взаимодействия заказчика и разработчика ИС, различных групп разработчиков ИС;
- содержит механизмы расширения и специализации базовых концепций языка

возможности

- строить модели на основе средств ядра, без использования механизмов расширения для большинства типовых приложений;
- добавлять при необходимости новые элементы и условные обозначения, если они не входят в ядро, или специализировать компоненты, систему условных обозначений (нотацию) и ограничения для конкретных предметных областей.

Версии UML

Версия	*Дата*
1.1	*ноябрь 1997*
1.3	*март 2000*
1.4	*сентябрь 2001*
1.4.2	*июль 2004*
1.5	*март 2003*
2.0	*июль 2005*
2.1	*формально не была принята*
2.1.1	*август 2007*
2.1.2	*ноябрь 2007*
2.2	*февраль 2009*
2.3	*май 2010*
2.4 beta 2	*март 2011*
2.5	*июнь 2015*
2.5.1	*декабрь 2017*


An OMG® Unified Modeling Language® Publication

OBJECT MANAGEMENT GROUP

UNIFIED MODELING LANGUAGE

OMG® Unified Modeling Language® (OMG UML®)

Version 2.5.1

OMG Document Number: formal/2017-12-05
Date: December 2017
Normative URL: https/www.omg.org/spec/UML/
Machine Readable:

UML 1.x и UML 2.0

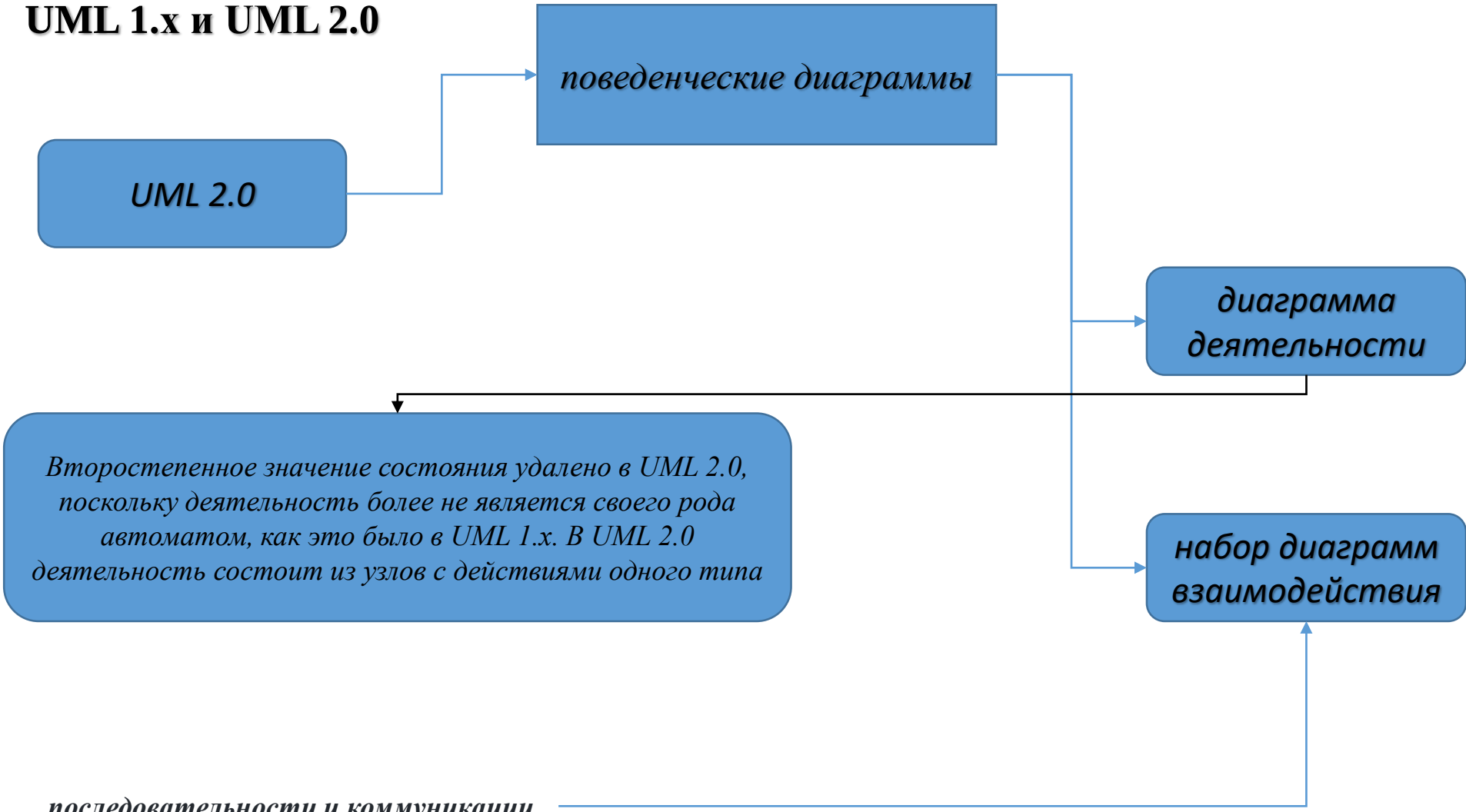


Диаграмма последовательности

В UML 2.0 представлено несколько новых функциональных возможностей для диаграммы последовательности:

•*Фрагменты обеспечивают более четкую семантику реализации поведения в диаграмме последовательности. Комбинированный фрагмент объединяет части диаграммы последовательности. Можно смоделировать отдельные потоки и просмотреть, как условия приводят к различным путям выполнения.*

Экземпляры взаимодействия допускают расщепления взаимодействий на многоразовые участки. Это позволяет использовать общие участки для нескольких взаимодействий.

•*В UML 1.x единственным возможным представлением циклов было применение условия цикла, записанного в Примечании. Примечание присоединялось к сообщению или набору сообщений, которые должны были исполняться до тез пор, пока условие цикла было верным. В UML 2.0 для циклов существует специальное представление.*

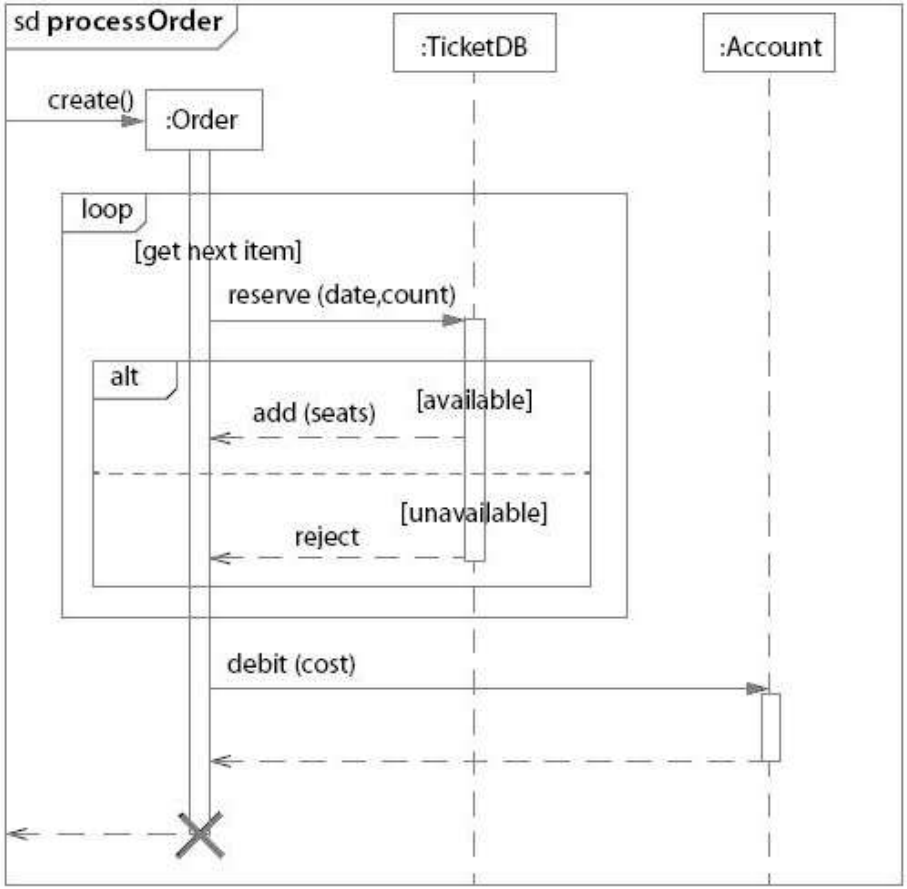
•*В UML 2.0 на диаграммах последовательностей может быть показано, как объекты создаются и уничтожаются.*

Выполнение экземпляра показывает текущий управляющий элемент, который исполняется объектом в некоторый момент времени при получении сообщения.

При наличии новых возможностей представления фрагментов - экземпляров взаимодействия и циклов - диаграммы последовательности можно использовать в двух формах:

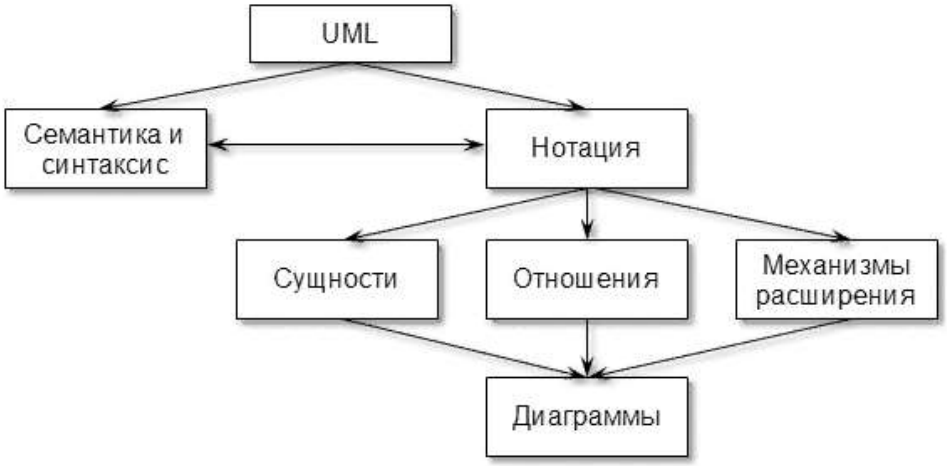
Отдельная форма*: подробно описывает определенный сценарий, документируя одно возможное взаимодействие без условий, ветвей или циклов. Эта форма применяется для представления одного сценария варианта. Различные сценарии одного и того же прецедента представляются на различных диаграммах последовательности. Инструменты моделирования, которые поддерживают семантику UML 1.x, допускают только такую форму представления.*

Общая форма: *описывает все возможные альтернативы в сценарии, используя преимущества новых функций в UML 2.0, таких как условия, ветви и циклы. Эту форму можно использовать для представления нескольких сценариев одного и того же варианта на одной диаграмме последовательности, там где это имеет смысл.*



2. Синтаксис и семантика основных объектов UML

Структура UML



- **синтаксис** (syntax), то есть определение правил составления конструкций языка;
- **семантика** (semantics), то есть определение правил приписывания смысла конструкциям языка;
- **прагматика** (pragmatics), то есть определение правил использования конструкций языка для достижения определенных целей.

Одним из назначений UML является:

- создание таких моделей, для которых возможна автоматическая генерация программного кода (или фрагментов кода) соответствующих приложений;
- автоматическое построение модели по коду готового приложения.

UML это не методология, это унифицированный язык визуального моделирования. UP – это методология.

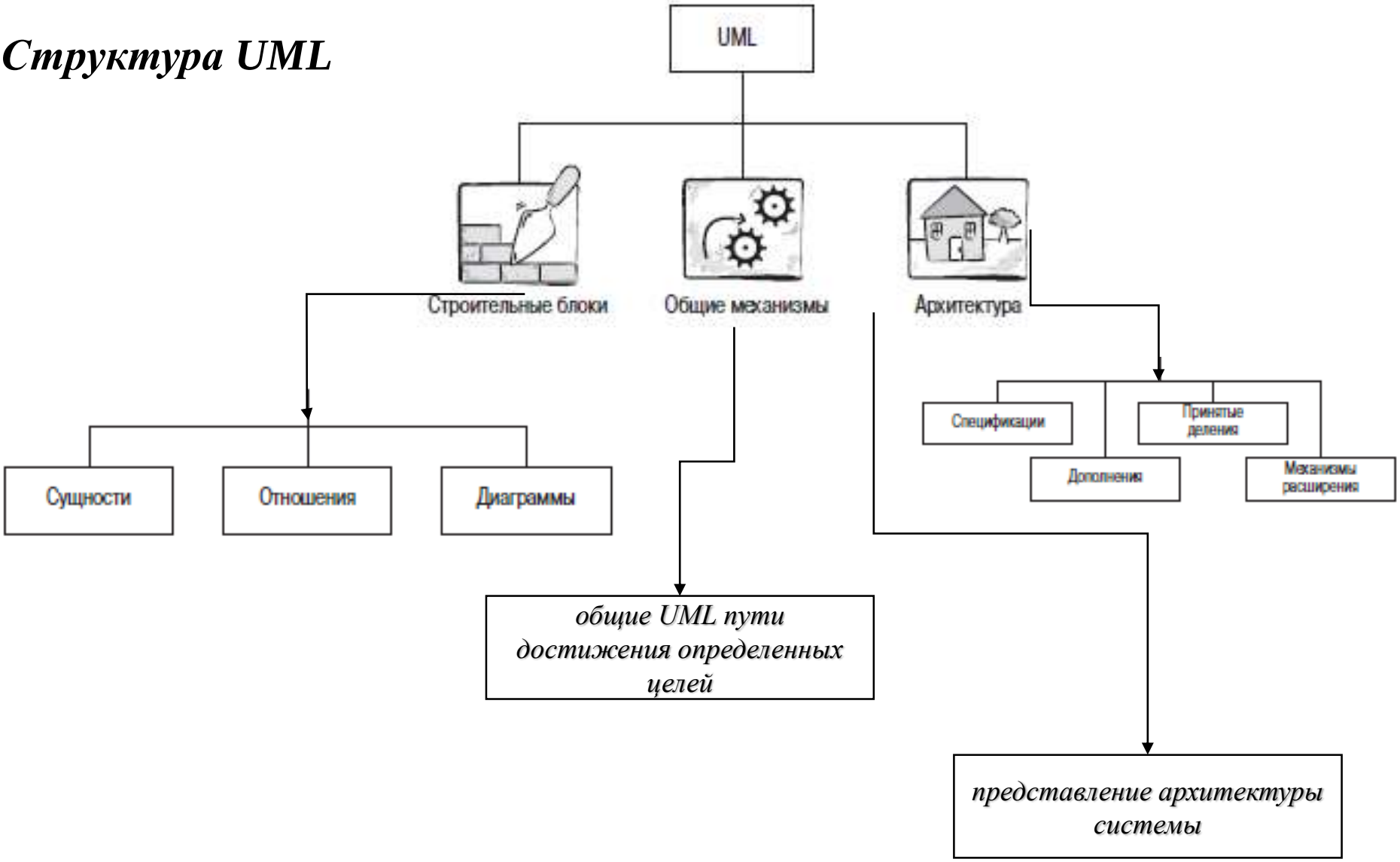
*Унифицированный процесс (**Unified Process, UP**) – это методология. Она указывает на исполнителей, действия и артефакты, которые необходимо использовать, осуществить или создать для моделирования программной системы.*

***UML** не привязан к какой- либо конкретной методологии или жизненному циклу. На самом деле он может использоваться со всеми существующими методологиями.*

***UP** использует UML в качестве базового синтаксиса визуального моделирования.*

*Основная идея **UML** – возможность моделировать программное обеспечение и другие системы как наборы взаимодействующих объектов.*

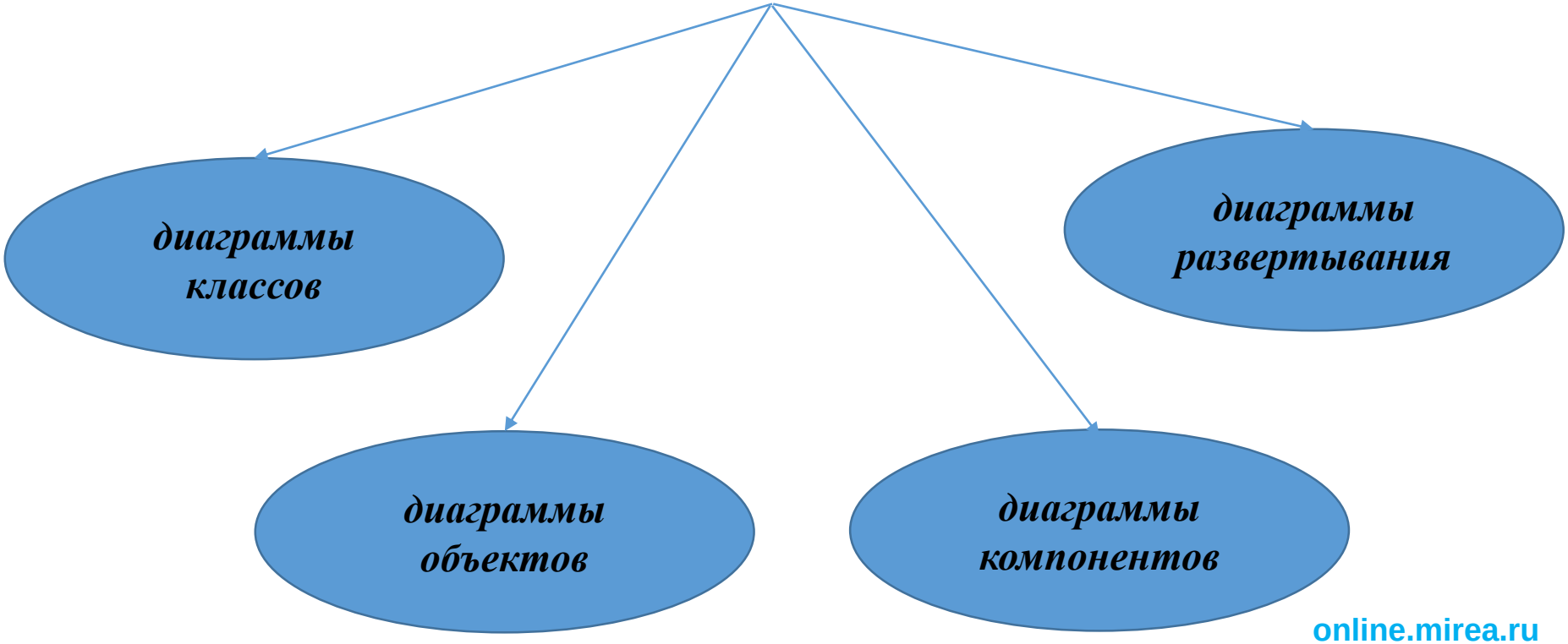
Структура UML



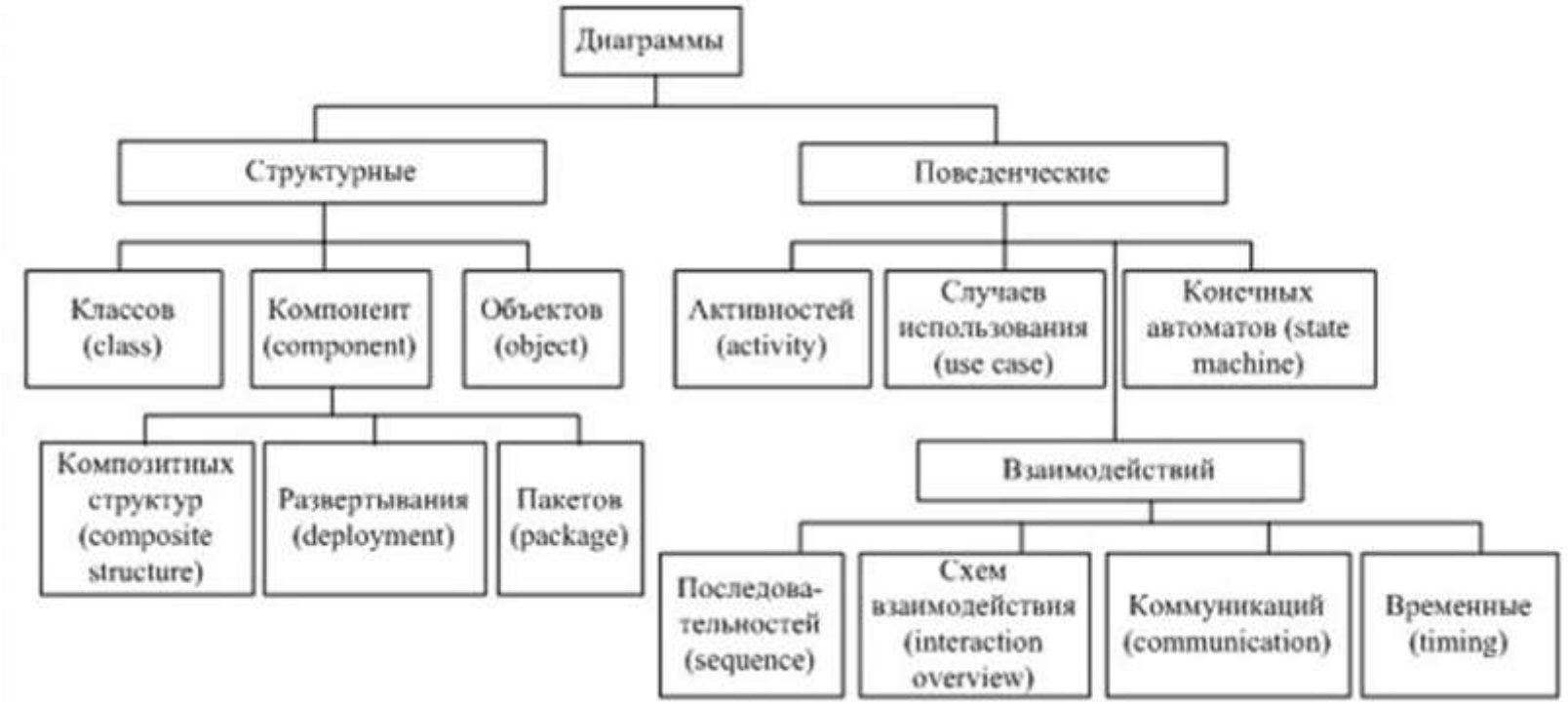
В UML модели есть два аспекта:

- *Статическая структура – описывает, какие типы объектов важны для моделирования системы и как они взаимосвязаны.*
- *Динамическое поведение – описывает жизненные циклы этих объектов и то, как они взаимодействуют друг с другом для обеспечения требуемой функциональности системы.*

Статические диаграммы



Иерархия типов диаграмм



1. Объект (object) — сущность, обладающая уникальностью и инкапсулирующая в себе состояние и поведение.

2. Класс (class) — описание множества объектов с общими атрибутами, определяющими состояние, и операциями, определяющими поведение.

3. Интерфейс (interface) — именованное множество операций, определяющее набор услуг, которые могут быть запрошены потребителем и предоставлены поставщиком услуг.

4. Кооперация (collaboration) — совокупность объектов, которые взаимодействуют для достижения некоторой цели.

5. Действующее лицо (actor) — сущность, находящаяся вне моделируемой системы и непосредственно взаимодействующая с ней.

6. Компонент (component) — модульная часть системы с четко определенным набором требуемых и предоставляемых интерфейсов.

7. Артефакт (artifact) — элемент информации, который используется или порождается в процессе разработки программного обеспечения.

8. Узел (node) — вычислительный ресурс, на котором размещаются и при необходимости выполняются артефакты.

9. Состояние (state) — период в жизненном цикле объекта, находясь в котором объект удовлетворяет некоторому условию и осуществляет собственную деятельность или ожидает наступления некоторого события.

10. Деятельность (activity) можно считать частным случаем состояния, который характеризуется продолжительными (по времени) не атомарными вычислениями.

11. Действие (action) — примитивное атомарное вычисление.

12. Вариант использования (use case) — множество сценариев, объединенных по некоторому критерию и описывающих последовательности производимых системой действий, доставляющих значимый для некоторого действующего лица результат.

13. Пакет (package) — группа элементов модели (в том числе пакетов).

UML обеспечивает большую гибкость при создании моделей. В частности, модели могут быть:

- *сокращенными – некоторые элементы присутствуют в заднем плане, но скрыты в той или иной диаграмме для упрощения представления;*

- *неполными – некоторые элементы модели могут быть полностью пропущены;*

- *несогласованными – модель может содержать противоречия.*

Классификатор – это абстрактное понятие, например тип банковского счета.
Экземпляр – конкретная сущность, например Ваш банковский счет.

3. Понятие модели и диаграмм

Система → *набор подсистем, организованных для достижения определенной цели и описываемых с помощью совокупности моделей, возможно, с различных точек зрения*

Модель → *некий (материальный или нет) объект, отображающий лишь наиболее значимые для данной задачи характеристики системы*

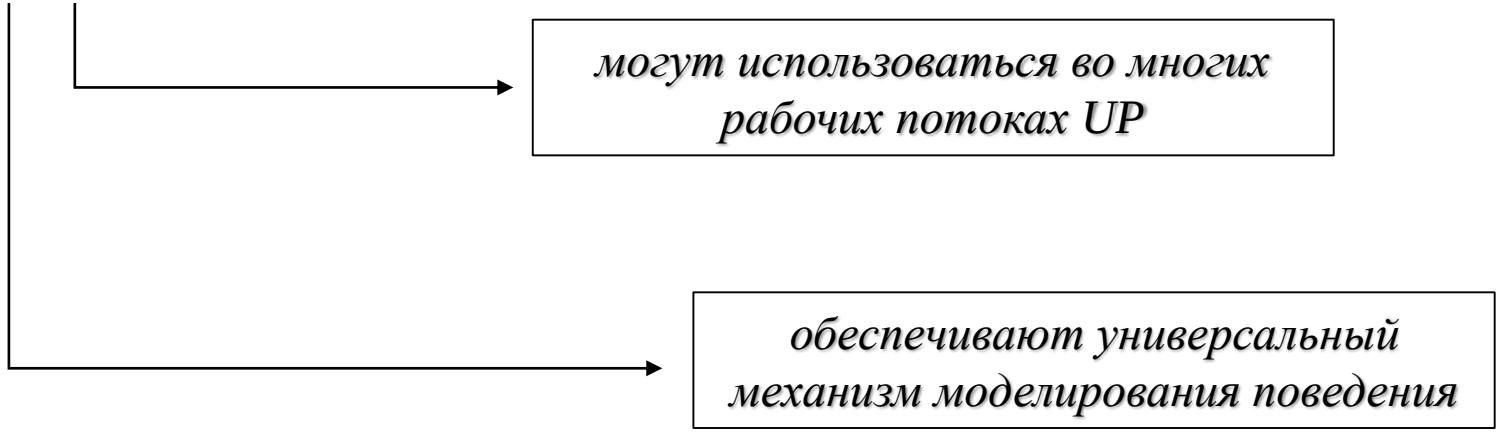
материальные и нематериальные, искусственные и естественные, декоративные и математические

Диаграмма → *графическое представление множества элементов*

с помощью диаграмм можно визуализировать систему с различных точек зрения

ни одна отдельная диаграмма не является моделью, лишь набор диаграмм составляет модель системы

Диаграммы деятельности



используются в следующих случаях:

В процессе анализа:

- *для графического моделирования потока прецедента. Такое представление является более понятным для заинтересованных сторон;*
- *для моделирования потока между прецедентами. При этом используется особая форма диаграммы деятельности – диаграмма обзора взаимодействий.*

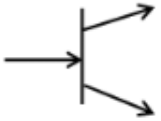
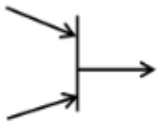
При проектировании:

- *для моделирования деталей операции;*
- *для моделирования деталей алгоритма.*
- *При моделировании деловой активности:*
- *для моделирования бизнес-процесса.*

Основными элементами диаграммы деятельности являются:

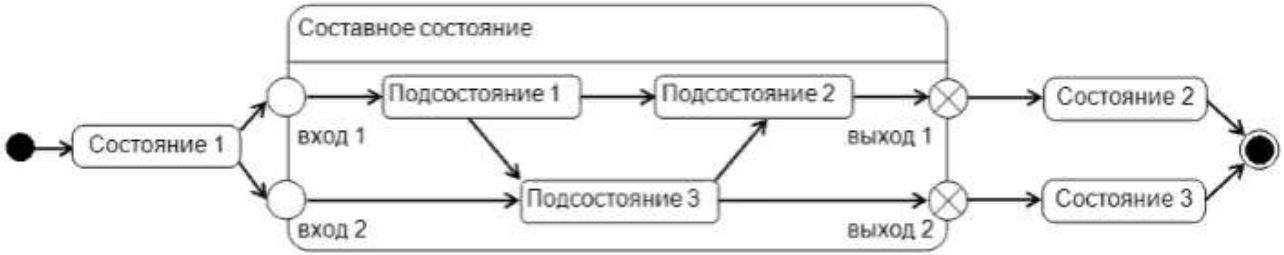
- исполняемые узлы;
- объекты;
- переходы;
- управляющие узлы;
- группирующие элементы.



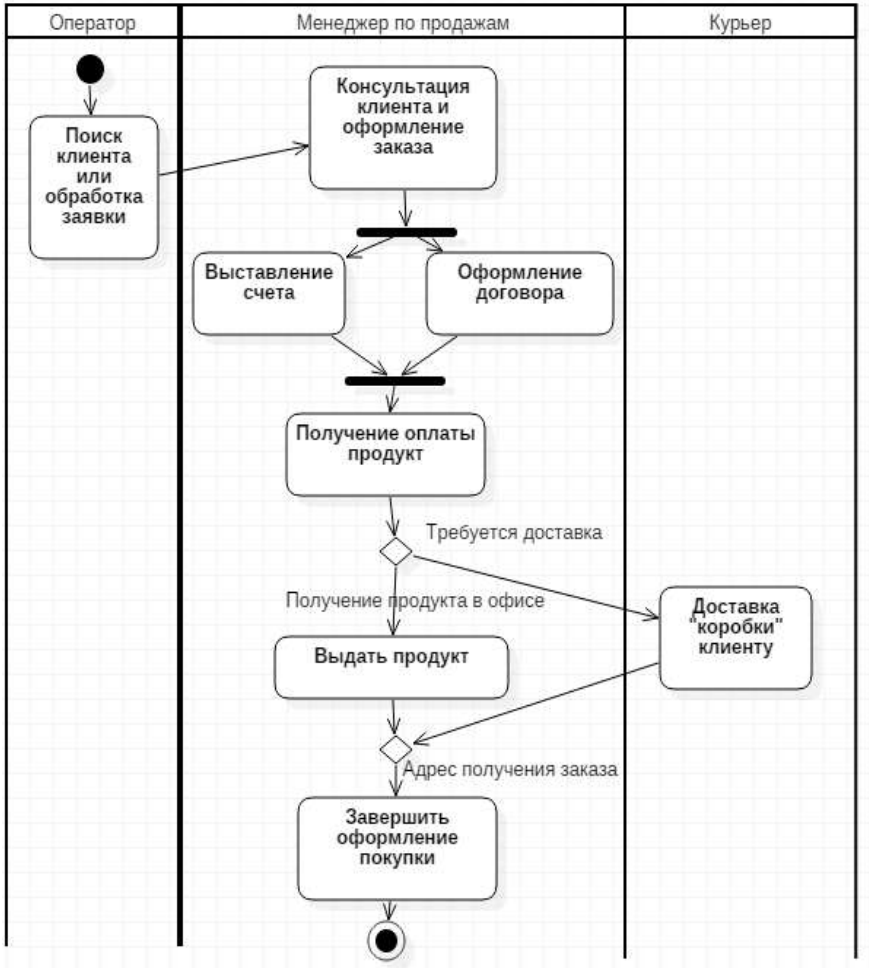
Обозначение	Наименование	
	на диаграмме автоматов	на диаграмме деятельности
●	Начальное псевдосостояние (англ. initial pseudostate)	Начальный узел (англ. initial node)
◉	Конечное состояние (англ. final state)	Завершение деятельности (англ. activity final)
⊗	Точка выхода (англ. exit point pseudostate)	Завершение потока (англ. flow final)
	Ветвление (англ. fork pseudostate)	Ветвление (англ. fork node)
	Соединение (англ. join pseudostate)	Соединение (англ. join node)
◇	Выбор (англ. choice pseudostate)	Слияние / решение (англ. merge / decision node)

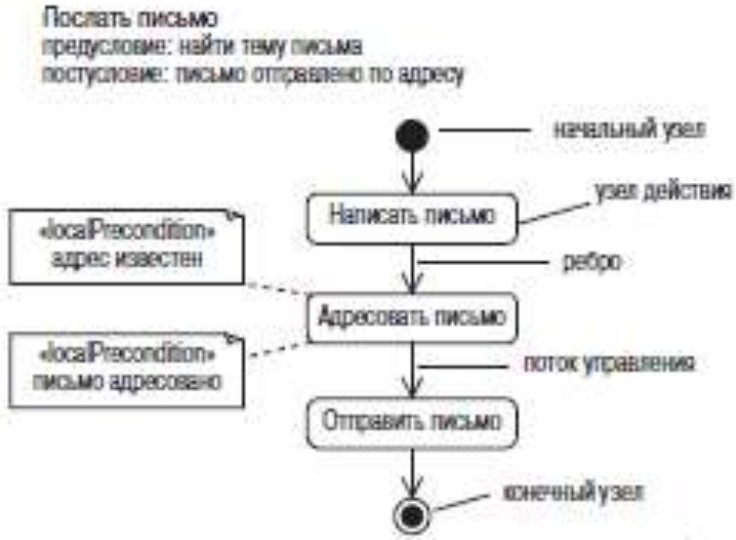


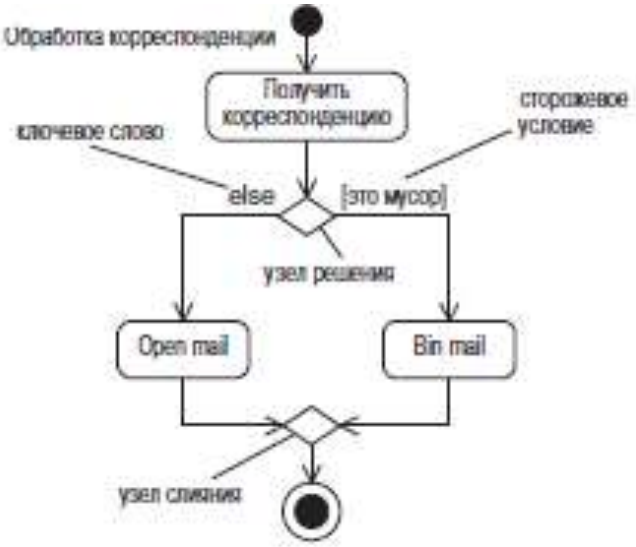
а) поток управления б) объектный поток в) поток прерывания или исключения



Activity Diagram:







Диаграммы деятельности основаны на технологии сетей Петри.

При разработке диаграммы деятельности следует придерживаться следующих правил:

1. Действие создается для каждой основной задачи. Заголовок каждого действия должен четко показывать, для чего оно необходимо.

2. Каждое действие заканчивается до начала следующего действия в потоке управления.

3. Количество пересечений линий следует минимизировать. При этом считается, что пересекающиеся линии не имеют логической связи друг с другом. Другими словами, потоки данных или управления в местах пересечений не меняют своего направления.

4. Если на диаграмме имеется ветвление / решение на параллельные или альтернативные потоки, то должно указываться и соответствующее соединение / слияние этих потоков.

5. При использовании альтернативных потоков каждый из них должен быть специфицирован с помощью сторожевого условия. Сторожевые условия не должны допускать одновременного срабатывания двух и более переходов.

Диаграммы *состояний* применяются для того, чтобы объяснить, каким образом работают сложные объекты.

Основные действия:

entry – действие, которое выполняется в момент входа в данное состояние;

exit- действие, которое выполняется в момент выхода из данного состояния;

do- выполняющаяся деятельность в течение всего времени, пока объект находится в данном состоянии;

defer – событие, обработка которого предписывается в другом состоянии, но после того, как все операции в текущем будут завершены.

Виды событий: *вызов, сигнал, таймер, изменение. Узлы: завершение, ветвление.*



СПЕЦИАЛЬНЫЕ ДИАГРАММЫ

Диаграмма объектов (object diagram) — является экземпляром диаграммы классов. На диаграмме объектов применяют один основной тип сущностей: объекты (экземпляры классов), между которыми указываются конкретные связи (чаще всего экземпляры ассоциаций).

Диаграмма внутренней структуры (composite structure diagram) используется для более подробного представления структурных классификаторов, прежде всего классов и компонентов.

Обзорная диаграмма взаимодействия (interaction overview diagram) является разновидностью диаграммы деятельности с расширенным синтаксисом: в качестве элементов обзорной диаграммы взаимодействия могут выступать ссылки на взаимодействия (interaction use), определяемые диаграммами последовательности.

Диаграмма синхронизации (timing diagram) представляет собой особую форму диаграммы последовательности, на которой особое внимание уделяется изменению состояний различных экземпляров классификаторов и их временной синхронизации.

Диаграмма пакетов (package diagram) — единственное средство, позволяющее управлять сложностью самой модели. Основные элементы нотации — пакеты и зависимости с различными стереотипами.

Диаграммы объектов *имеют вспомогательный характер – по сути это примеры (можно сказать, дампы памяти), показывающие, какие имеются объекты и связи между ними в некоторый конкретный момент функционирования системы.*

1. (экземпляры классов)
2. (чаще всего экземпляры ассоциаций)

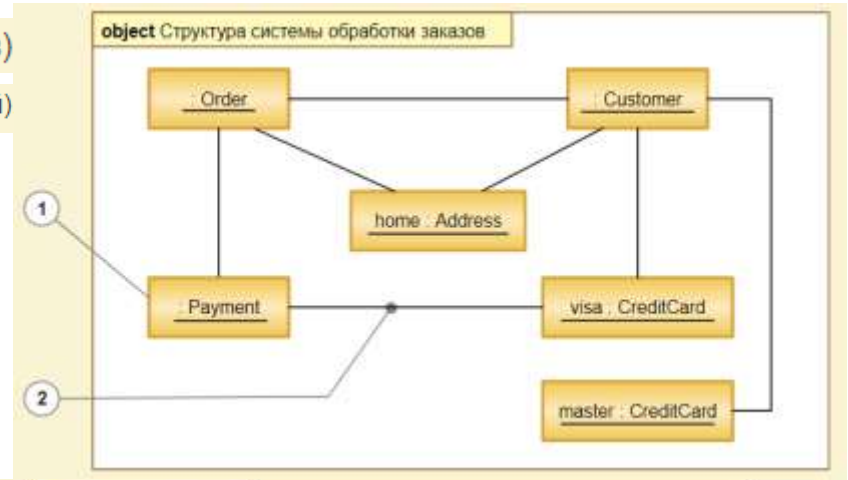
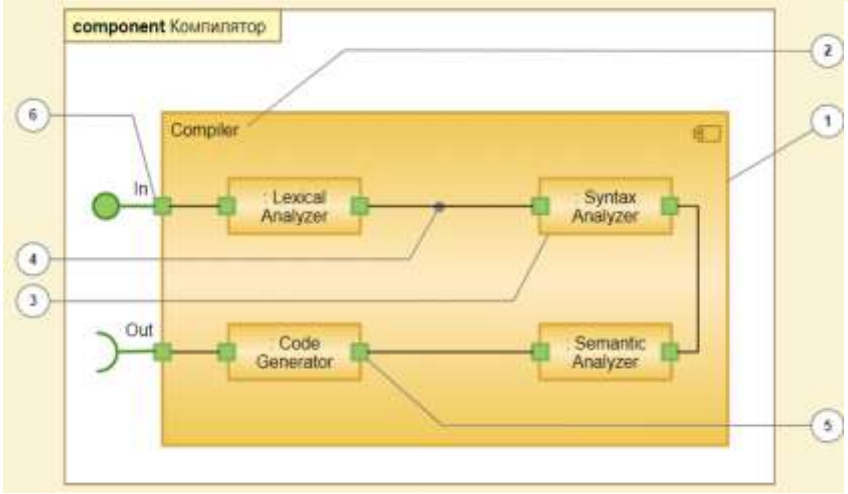


Диаграмма внутренней структуры *(composite structure diagram) используется для более подробного представления структурных классификаторов, прежде всего классов и компонентов.*



Структурный классификатор изображается в виде прямоугольника -1, в верхней части которого находится имя классификатора -2. Внутри находятся частии (parts) 3. Частей может быть несколько. Каждая часть является экземпляром некоторого другого классификатора. Части могут взаимодействовать друг с другом. Это обозначается с помощью соединителей (connectors) - 4 различных видов. Место на внешней границе части, к которому присоединяется соединитель, называется портом (port) -5. Порты располагаются также на внешней границе структурного классификатора- 6, обеспечивая ему связь с внешним миром.

Обзорная диаграмма взаимодействия

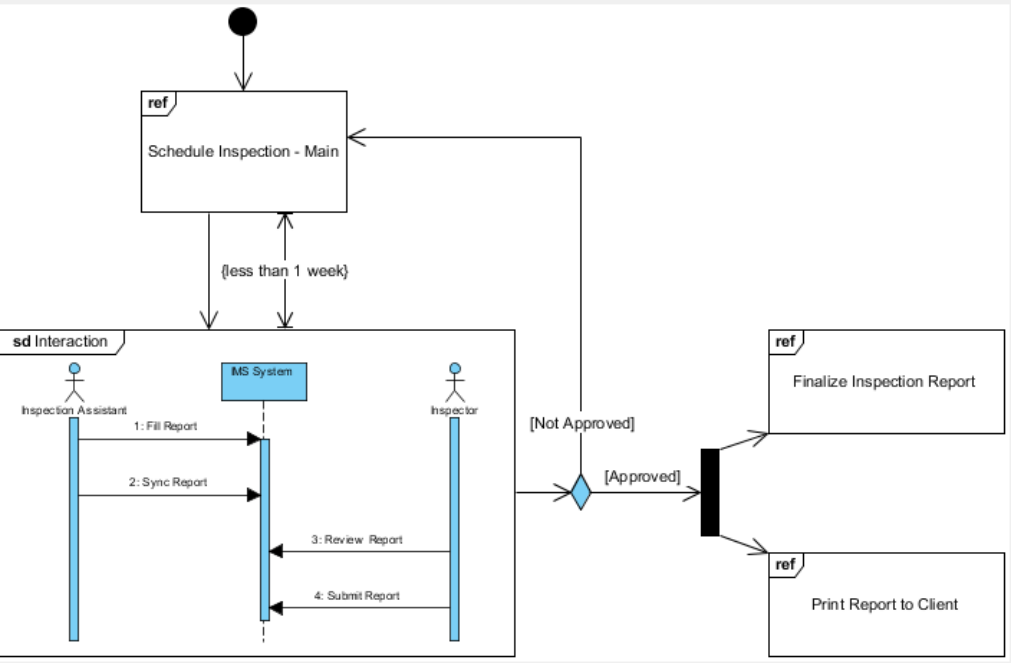
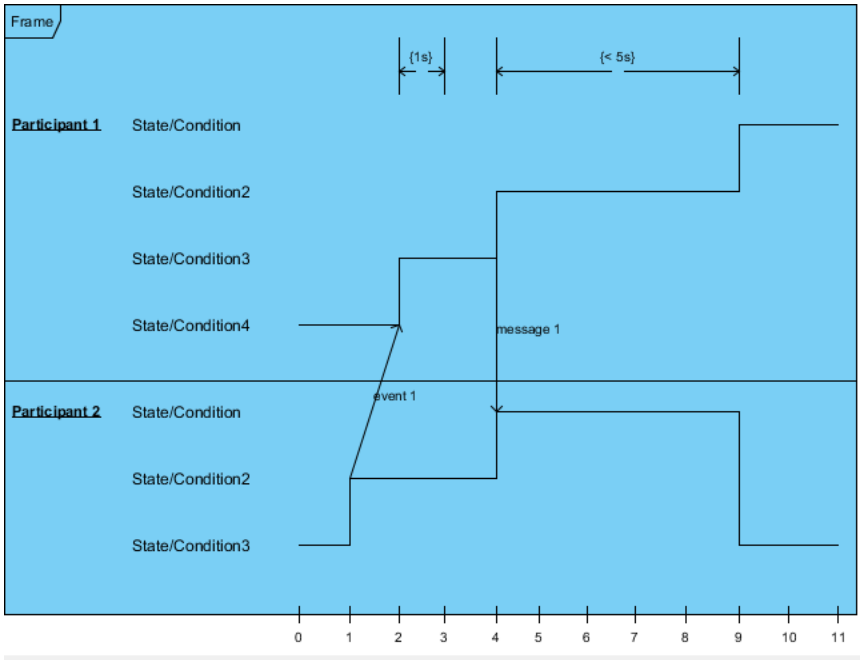
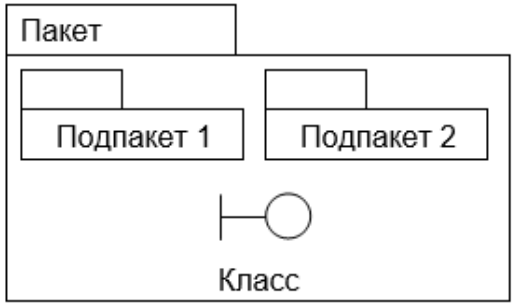


Диаграмма синхронизации

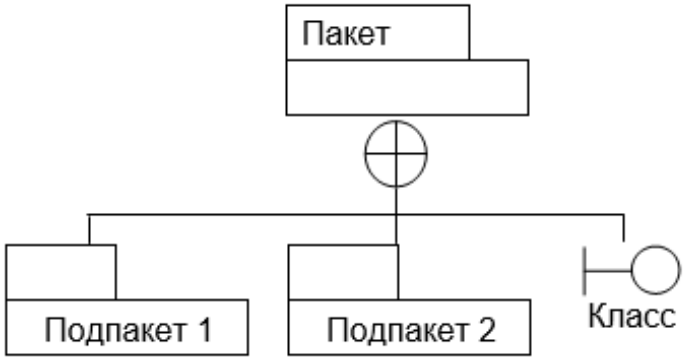




В результате разработки диаграмм классов анализа, кооперации и последовательности определяется набор основных классов, определяющих внутреннюю архитектуру системы.

На стадии анализа выявляются основные характеристики классов (атрибуты) и их поведение (методы).

Выявленные классы должны быть распределены по **пакетам** (package, англ.) – сущностям, используемых для семантической группировки других сущностей.



Допускается (как правило, имеет место) вложенность пакетов, т.е. пакет может состоять из подпакетов, подпакеты – из подподпакетов и т.д. Содержимое пакета может отображаться двумя способами.

Диаграмма пакетов – единственное средство, позволяющее управлять сложностью самой модели. Основные элементы нотации – пакеты и зависимости с различными стереотипами, применяемые на диаграмме

Спасибо за внимание!